

视频课程笔记

Matt Pocock 的 Agentic Engineering Workflow

把代理编程变成可排队、可隔离、可评审、可复盘的工程 workflow

David Ondrej 访谈整理 & Codex

2026 年 7 月 1 日



视频频道: David Ondrej

发布日期: 2026 年 6 月 18 日

视频时长: 01:02:24

视频链接: <https://www.youtube.com/watch?v=nQwJVHctDDY>

字幕来源: YouTube 英文字幕, 按时间轴重组为中文教学笔记。

关键结论

导读：本笔记的中心答案 Matt Pocock 这场访谈的主线可以先压成一句话：**agentic engineering** 指围绕代理建立可验证工作流的工程实践，它的耐久优势来自模型之外的运行支架。这里先约定几组术语：**harness** 是围绕模型运转的提示词、工具、测试、文档和流程；**procedure skill** 是把可重复操作写成可复用步骤；**queue** 是待办任务进入、排序、领取和回收的队列；**sandbox** 是限制文件、网络、凭据和运行环境的隔离边界；**review feedback** 是把评审发现回流到代码、测试和流程的证据；**AFK agent** 是人在离开键盘时仍能在限定任务内运行的代理；**AX** 是 **Agent Experience**，即代理在代码库里理解任务、调用命令、验证结果和产出可审查变更的体验。基于这些定义，强模型提供战术执行力，工程师真正能持续改进的是运行支架、流程技能、队列组织、隔离边界、评审回流、代码库结构和代理体验。读者读完这份笔记后，应能把一个模糊的代理编程愿望改造成可排队、可隔离、可评审、可复盘的工程工作流。

背景知识

阅读路径 前两章建立总论：运行支架是可控乘数，人类转向 **strategic programming**。第三、四章用 **teach skill** 和技能分类说明经验怎样写成可复用流程。第五章进入离线代理队列的操作系统。第六章处理评审、产品判断与代理体验。第七章把全片压回一个行动序列：空白基线、观察、逐步加回流程技能，再把边界清楚的实现任务委派给离线代理。

目录

1 模型之外：harness 才是可控杠杆	2
1.1 问题：只盯着 model，会把工程杠杆交给别人	2
1.2 主张：agentic output 来自多个因子的相乘	2
1.3 机制：好的 harness 降低探索成本和返工成本	3
1.4 反例压力：bitter lesson 并没有取消 harness	4
1.5 落地：把 harness 拆成每天能改的一组清单	4
1.6 本章小结	5
2 战略编程：tactical programming 交给代理，人类负责战局	5
2.1 问题：当 tactical programming 变便宜，人类还剩什么价值	5
2.2 主张：人的杠杆从敲代码转向 shaping work	6
2.3 机制：有效委派需要把上下文压成可执行边界	7
2.4 例子：把“让代理改功能”改写成战略任务	7
2.5 技能上限：Domain Skill 决定代理能被带到哪里	8
2.6 本章小结	9
3 teach skill：把学习变成有状态的工作流	9
3.1 问题：学习代理很容易退化成资料搬运工	9
3.2 想法：先问 mission，再决定 lesson	9
3.3 机制：从 mission.md 到 learning record	10
3.4 例子：Git lesson 如何服务真实应用	12

3.5	本章小结	12
4	写 skill 的工程判断: procedure skill 优先, 谨慎使用 ability skill	13
4.1	问题: skill 多了以后, 真正稀缺的是调用权和上下文预算	13
4.2	想法: procedure skill 把判断留在人手里	13
4.3	机制: ability skill 的成本来自自动发现	14
4.4	例子: 从 grill me 到 PRD 和 issue splitting	15
4.5	知识、技能、智慧: skill 能封装前两者, 难以替代第三者	16
4.6	本章小结	17
5	AFK 工作流: sandbox、queue 与 human-in-the-loop checkpoints	17
5.1	问题: 把 agent 放出去干活, 首先会遇到边界问题	17
5.2	例子零: Fable/Cursor 的安全教训要写回 harness	17
5.3	想法: 把 agentic work 优先看成 queue	18
5.4	机制: 从 issue label 到 explore、implement、review	19
5.5	例子一: Sandcastle 把 AFK agent 放进 sandbox	20
5.6	例子二: GitHub Actions review agents 把检查接进 PR	20
5.7	human-in-the-loop checkpoints: 随着信心增长向右移动	21
5.8	操作 takeaway: 把 AFK 做成一套可回收的生产线	22
5.9	本章小结	22
6	评审、产品与 AX: 保留判断力, 减少低价值人工成本	23
6.1	问题: 自动化越强, 越要保留人的判断位置	23
6.2	核心判断: 把 review 设计成反馈系统, 而非人工瓶颈	23
6.3	机制: 自动合并需要风险分层、抽样审计与回滚计划	24
6.4	例子与证据: 更丰富的 review artifacts 会改变人的工作方式	25
6.5	产品判断: AI 加速实现, 产品方向仍来自真实世界	26
6.6	AX 与 DX: 同一个 codebase, 两个使用者	26
6.7	团队含义: 资深经验与 AI-native 操作力要合流	27
6.8	本章小结	28
7	总结与延伸: 从空白支架开始, 逐步加回 procedure skills	28
7.1	问题: harness 越厚, 越难知道什么真的有效	28
7.2	核心公式: Agentic Output 来自四个乘数	28
7.3	机制: 从空白 baseline 到可复用 procedure skills	29
7.4	把个人实践变成团队技能	30
7.5	最终综合: 战略编程是持续改善乘数	30
7.6	读者今天可以做的三件事	31
7.7	本章小结	31

1 模型之外：harness 才是可控杠杆

1.1 问题：只盯着 model，会把工程杠杆交给别人

这场访谈一开头就把矛盾放在桌面上：很多人把注意力押在最新的 model 上，期待换一个更强模型就能突然获得巨大产能；Matt 的判断更冷静，他认为真正可日常改进、可复用、可沉淀的部分，是围绕模型运转的 harness。这里的 harness 可以译作“代理运行支架”：它包含提示词、技能、工具、沙箱、代码库结构、测试、文档、CI、评审流程与运行环境。

这个判断对工程团队很刺耳。模型能力当然重要，强模型像更好的发动机，能让同一辆车跑得更快；问题在于，普通团队无法控制下一代模型什么时候发布、价格怎样、延迟多高、上下文窗口多大。团队能直接控制的，是车架、悬挂、轮胎、维修制度和驾驶策略。把这个类比放回代理编程，model 是发动机，harness 是让发动机可控地输出工程成果的整套车辆系统。

关键结论

核心定义：harness 是可控乘数 harness 指模型外部的工程运行系统：提示、技能、工具、代码库、测试、文档、权限边界、沙箱、CI、评审与反馈。它的价值来自稳定性、成本控制、可审查性和团队协作能力：同一个 model 放进更好的支架里，输出会更可靠。

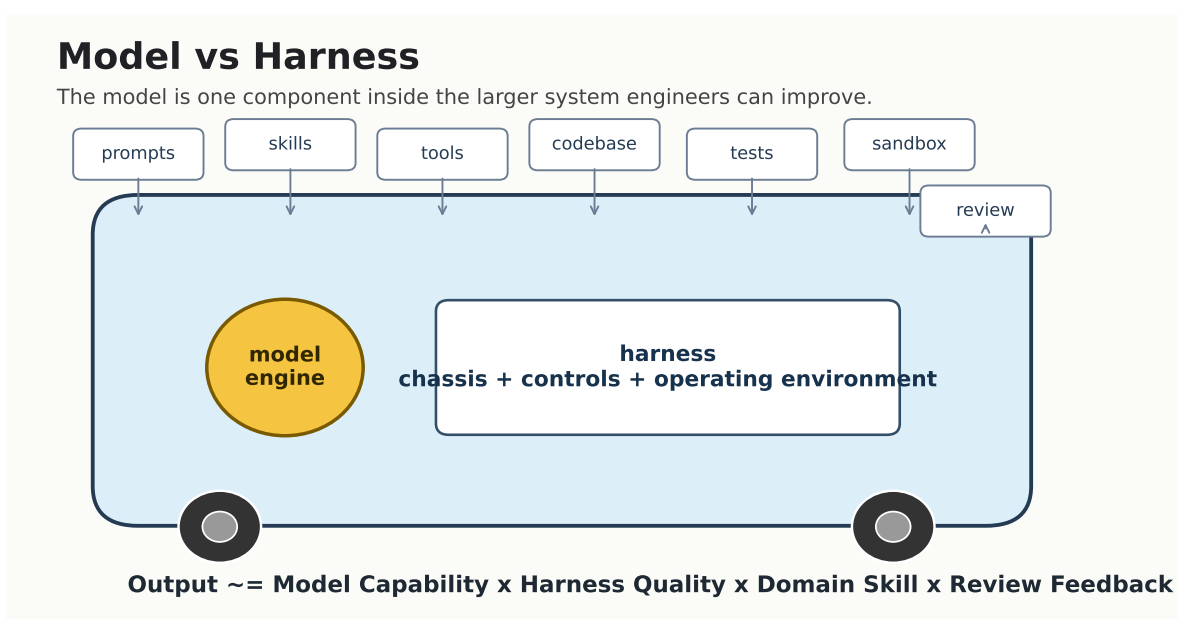


图 1：模型只是系统里的发动机；harness 才是工程师每天能持续调优的整套运行支架。图根据 00:27:45–00:28:31 的 Formula 1 类比和全局乘法公式重绘。

1.2 主张：agentic output 来自多个因子的相乘

这份 PDF 后面所有章节都可以压缩成一个乘数模型。它属于工程判断框架，重点在揭示因子之间的联动：任何一个关键因子过低，整体输出都会被拖住。

$$\text{Agentic Output} \approx \text{Model Capability} \times \text{Harness Quality} \times \text{Domain Skill} \times \text{Review Feedback}$$

- Agentic Output：代理系统最终交付的工程成果，包括可运行代码、通过测试的改动、清晰的文档、可审查的 PR，以及能进入下一轮迭代的经验。

- **Model Capability:** 原始 model 能力，包括推理、编码、工具调用、视觉理解、电脑操作和上下文处理能力。
- **Harness Quality:** harness 的质量，包括提示、技能、沙箱、代码库设计、文档、测试、CI、权限、运行环境和评审机制。
- **Domain Skill:** 人类对领域的理解、实践技能和判断力。Matt 后面把它压缩成一句话：人的技能会成为 AI 输出的上限。
- **Review Feedback:** 人类与自动化检查点提供的反馈，包括代码评审、测试失败、运行日志、用户反馈、安全审查和复盘记录。

乘法关系的含义很直接：某个因子接近零时，其他因子再强也很难弥补。一个强 model 在混乱代码库里会花大量 token 猜入口、读重复逻辑、绕过脆弱测试；一个干净的 harness 能让较便宜的模型完成同样任务，因为它少走弯路，少被环境反噬。

背景知识

直觉类比：赛车发动机与完整赛车 若把 model 当作发动机，harness 就是底盘、刹车、轮胎、燃油系统、维修流程和赛道策略。只升级发动机可能提高峰值速度；底盘和流程差时，发动机的额外功率会变成打滑、失控和维修成本。代理工程也是同理：模型越强，越需要能承接它输出的工程系统。

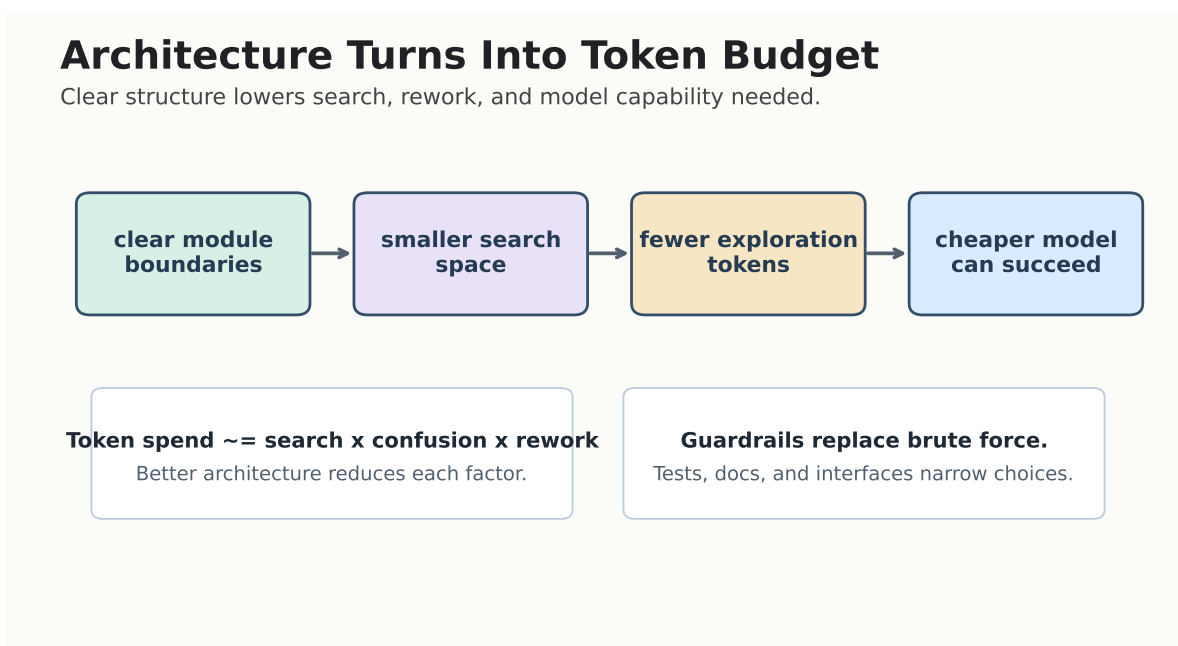


图 2: 代码库结构会转化为 token budget：模块边界越清楚，代理搜索空间越小，返工越少，较便宜的模型也更容易完成任务。图根据 00:32:22–00:32:57 的 token spend 讨论重绘。

1.3 机制：好的 harness 降低探索成本和返工成本

Matt 提到一个很实在的问题：如何优化 token spend？他的回答绕开折扣套餐，直指代码库可修改性。这个说法背后的机制可以拆成四步。

1. **入口更清楚：**文档、命名、模块边界和测试命令清晰时，代理不用花大量 token 猜“应该改哪里”。

2. **变更面更小**：模块边界稳定时，任务可以被切成小块，代理在有限上下文里也能完成局部改动。
3. **反馈更快**：测试、lint、类型检查和 CI 能把错误尽早暴露，减少长时间偏航。
4. **审查更便宜**：PR 小、证据清楚、运行日志齐全时，人类评审可以看机制和风险，而非重新侦探整段实现。

这就是 **harness** 的真实含金量：它把“聪明模型才能勉强做”的任务，改造成“普通模型也能稳定做”的任务。换句话说，架构清晰度本身就是算力节省器。一个易变更代码库给代理提供了窄通道、明确边界和及时反馈，代理花在探索、误读和返工上的 token 会减少。

常见误区

常见误区：工具轮换会掩盖基本功不足 频繁追最新工具、最新模型、最新一键生成平台，容易制造“自己在进步”的幻觉。若代码库仍然难以理解，测试仍然脆弱，任务仍然缺少边界，模型升级只会把混乱执行得更快。Matt 的建议是回到长期有效的软件工程基本功：清晰模块、可运行测试、恰当文档、可审查变更和稳定反馈。

1.4 反例压力：bitter lesson 并没有取消 harness

主持人对 Matt 提出一个合理挑战：机器学习里有一个著名观点叫 **bitter lesson**，大意是通用计算规模经常胜过人工设计的局部技巧。把它放到代理工程里，似乎可以推出一个懒惰结论：与其打磨 **harness**，不如等更强模型。

这个挑战值得认真对待。若未来 **model** 大幅进步，它确实会减少某些提示技巧和局部补丁的价值；更强发动机会让同一辆车更快。Matt 的回应更像工程师的操作原则：他不试图预言下一代模型，而是把工作空间和 **harness** 做得尽量模型无关，把注意力放在过去十几年、几十年一直有效的工程基本功上。

背景知识

bitter lesson 对本章的启发 **bitter lesson** 提醒读者别把一次性的技巧神化。某些“专门哄当前模型”的提示模板，可能会随模型迭代迅速贬值。更耐久的做法，是改善模型进入代码库、理解任务、执行改动、接受反馈的路径。这样的 **harness** 对旧模型、新模型和未来模型都有迁移价值。

所以本章的结论需要准确表述：模型仍然重要；**harness** 是团队更能直接改进的内部杠杆。团队应该使用足够好的模型，同时把主要学习投入放在能沉淀下来的系统资产上：代码结构、任务拆解、技能库、测试体系、评审证据和复盘机制。

1.5 落地：把 harness 拆成每天能改的一组清单

若把 **harness** 当作抽象口号，它很快会变成空话。更可执行的做法，是把它拆成一组可检查资产。

- **提示与任务描述**：任务有目标、边界、输入、输出、验收标准和禁止事项。
- **技能与流程**：高频工作被写成可复用 **skill**，复杂工作有明确阶段和检查点。
- **代码库结构**：模块边界清楚，命名稳定，入口文件和测试命令容易找到。

- **环境与沙箱**：代理有可运行环境，也有权限边界，避免为了完成任务而碰到无关资源。
- **测试与 CI**：代理能快速得到机器反馈，人类能看到结果证据。
- **评审与复盘**：每次失败都能沉淀为更好的提示、测试、文档或任务模板。

这套清单也解释了为什么 harness 是“可控杠杆”。团队无法每天训练一个新模型；团队可以每天让一个测试更可靠、一个任务模板更清楚、一个模块边界更干净、一个评审记录更有复用价值。

关键结论

本章压缩在 Matt 的框架里，先进模型提供战术执行力，harness 决定这股执行力能否稳定变成工程产出。真正值得长期投入的，是能让任何模型更好工作的环境：清晰代码、好任务、好测试、好技能、好评审。

1.6 本章小结

本章建立了整份笔记的总答案：代理工程的耐久优势来自围绕 model 设计 harness。乘数公式提醒读者，输出由模型能力、运行支架、领域技能和评审反馈共同决定。Matt 的 Formula 1 类比说明了为什么只关注发动机会漏掉整辆车；token spend 的例子则说明，代码库结构本身会影响代理成本和成功率。下一章会接着回答人的位置：当代理接管大量 tactical programming，人类应该把主要精力投入 strategic programming。

2 战略编程：tactical programming 交给代理，人类负责战局

2.1 问题：当 tactical programming 变便宜，人类还剩什么价值

Matt 用 John Ousterhout 在 *A Philosophy of Software Design* 中的区分，引出这场访谈最关键的人类角色问题：tactical programming 是地面层面的编码工作，包括写语法、改文件、修 bug、处理提交和追逐眼前问题；strategic programming 是更高层的工程判断，包括架构、速度、模块边界、测试策略、文档、任务拆解、路线图和委派。

在 Matt 的强表述里，AI 已经大量吞掉 tactical programming。这句话需要谨慎处理。它表达的是他的操作判断：对于许多日常实现任务，代理已经足够便宜、足够快、足够适合承担执行层劳动。它并不意味着所有代码都能自动生成，也不意味着人类可以放弃技术判断。相反，这句话把人的工作推向更难层面：决定要打哪场仗、怎么布阵、如何设边界、怎样检查结果。

背景知识

John Ousterhout 的 tactical/strategic 区分 John Ousterhout 讨论的核心张力是短期推进和长期系统质量之间的取舍。用在代理时代，tactical programming 像现场修路：快速填坑、让车先过；strategic programming 像城市规划：道路网络、排水系统、路标规则和维护制度。代理很适合承担大量现场施工，人类仍要负责规划是否合理。

Tactical vs Strategic Programming

Agent leverage rises when humans shape the work before tactical execution begins.

Tactical programming	Strategic programming
Unit of work edits, syntax, bugs, commits	architecture, boundaries, scope
Core question How do I make this change?	Which change should exist?
Agent role fast implementer	delegatee inside a plan
Human leverage review local output	shape task, interface, tests
Main failure buggy patch	wrong problem or vague task

Delegation rule
design hard parts up front; agent handles scoped tactical work

图 3: tactical programming 关注局部实现，strategic programming 关注任务选择、边界、接口和测试；代理越强，人类越需要把前置设计做清楚。图根据 00:00:55–00:02:45 的论述重绘。

2.2 主张：人的杠杆从敲代码转向 shaping work

如果第一章的关键词是 **harness**，本章的关键词就是“塑造工作”。Matt 的观点可以压缩成一句：AI 让战术劳动变得充裕，人类价值转向战略编程。这里的“战略”并不玄，它落在很具体的工程动作上。

- 选择正确问题：判断哪个用户问题、技术债或产品假设值得进入队列。
- 设计硬部分：先把最容易出事故的接口、数据流、安全边界和状态模型想清楚。
- 缩小任务范围：把大目标切成代理能独立完成、能测试、能审查的小任务。
- 明确模块接口：让代理知道哪些文件可改、哪些契约要保持、哪些行为要证明。
- 建立反馈路径：用测试、类型检查、截图、日志、PR 描述和人工评审收敛结果。

关键结论

核心转移：从写代码到塑造工作 代理时代的人类优势主要来自“把工作变成好任务”的能力。好任务有明确目标、局部边界、必要背景、验收方式和风险提示；坏任务把模糊愿望丢给代理，然后期待它自动理解产品、架构、安全和用户语境。

这个转移也解释了为什么 **strategic programming** 没有因为 AI 出现而消失。Matt 说，委派给 AI 和委派给初中级开发者在原则上相似：任务仍需要提前设计，接口仍需要清楚，测试仍需要可信，文档仍要足够指路。区别在于代理可以更便宜地并行执行战术层任务，人的战略失误也会被更快放大。

2.3 机制：有效委派需要把上下文压成可执行边界

实践中的委派不是一句“实现这个功能”。对代理来说，真正有用的是一份边界清楚的执行包。它至少包括六个部分。

1. **任务目标**：要解决哪个问题，完成后用户或系统状态发生什么变化。
2. **输入材料**：相关文件、设计文档、错误日志、截图、接口定义、已有测试和历史决策。
3. **允许范围**：代理可以编辑哪些文件，可以运行哪些命令，哪些目录属于只读材料。
4. **接口契约**：外部 API、数据结构、模块边界和兼容性要求。
5. **验收标准**：测试命令、构建命令、浏览器验收、性能指标、安全检查或人工检查点。
6. **回报格式**：改了什么、为什么这样改、如何验证、还有哪些风险和未解问题。

这六项把人的 **strategic programming** 变成代理可消费的 **harness**。它们的共同目标，是减少代理在错误空间里乱试的机会。任务越清晰，代理越像拿到工单的工程师；任务越模糊，代理越像在陌生城市里凭感觉找路。

常见误区

强主张的边界 “AI 吃掉了 **tactical programming**” 应理解为 Matt 的工作流判断，而非对所有行业、所有代码、所有团队的普遍定律。高风险系统、模糊产品问题、复杂遗留系统和强合规场景仍需要严格的人类设计与审查。把战术劳动交给代理，前提是人类已经给出边界、证据和回收机制。

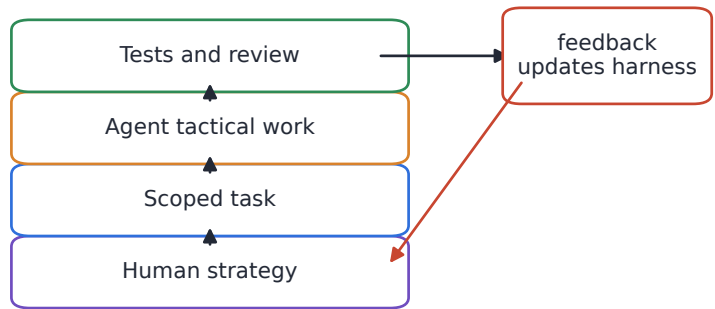
2.4 例子：把“让代理改功能”改写成战略任务

假设团队想让代理修复一个设置页的保存 bug。低质量委派会写成：“修一下设置页保存问题。”这会迫使代理先猜测问题位置、复现方式、风险边界和验收标准。更好的委派会按战略编程的方式组织：

- **目标**：用户修改通知偏好后，刷新页面仍能看到最新设置。
- **上下文**：相关入口是 **settings** 路由、偏好 API、当前失败日志和已有表单测试。
- **边界**：只处理持久化和表单状态同步，不改通知发送策略。
- **接口**：保持 API 响应结构不变，新增字段需要迁移说明。
- **验证**：添加或更新回归测试，运行设置页测试，必要时提供浏览器截图。
- **回报**：说明根因、改动点、验证命令和剩余风险。

这个例子体现了 Matt 所说的“先设计 **hard parts**”。人类并没有替代理写每一行代码；人类先把问题边界、接口和验证机制定下来。代理承担 **tactical programming**：读文件、改代码、跑测试、修失败、形成 PR。人类承担 **strategic programming**：判断这个 bug 值不值得修、风险在哪里、哪些契约不能破坏、什么时候可以合并。

Strategic delegation stack



The human keeps judgment; the agent handles bounded execution.

图 4: 战略委派层级: 人类保留判断, 先把目标压缩成有边界的任务; 代理承担战术执行; 测试与评审把反馈写回 harness。图根据 00:00:55–00:04:30 的委派讨论重绘。

2.5 技能上限: Domain Skill 决定代理能被带到哪里

Matt 在早段访谈里把个人技能描述为 AI 的乘数, 后面又强调 “skills are the ceiling”。这句话和第一章公式里的 Domain Skill 对上了: 代理能生成很多文本和代码, 但人类要知道什么是好设计、什么是坏抽象、什么风险值得提前处理、什么测试能证明行为。

现场对话

概念锚点 (00:04:13–00:04:15) **Matt**: “skills are the ceiling.”

这句短语之所以值得保留, 是因为它把 “学 AI 工具” 和 “学软件工程” 重新接起来。只会调用工具的人, 容易把战略问题也外包出去; 懂领域的人, 能把代理变成放大器。更好的老师能用 AI 教得更好, 更好的工程师能用代理构建更可靠的系统, 更懂产品的人能让代理少做无效功能。

关键结论

委派检查清单 有效代理委派会保留人的战略判断, 并把已经成形的判断包装成可执行任务:

- **清楚目标**: 先定义要解决的问题, 以及完成后读者、用户或代码库会出现什么可观察变化。
- **限定上下文**: 只给代理当前任务需要看的材料, 避免把无关历史和噪声塞进上下文。
- **稳定接口**: 提前说明不能破坏的 API、数据契约、测试边界和用户承诺。
- **快速反馈**: 用测试、类型检查、审查清单或小步提交让错误尽早暴露。

这四项构成检查顺序: 目标不清时先回到问题定义, 上下文过宽时先裁剪材料, 接口不稳时先补契约, 反馈太慢时先补验证路径。

2.6 本章小结

本章把人的位置讲清楚：在 Matt 的框架中，代理承担越来越多 **tactical programming**，人类更应该强化 **strategic programming**。John Ousterhout 的区分提供了概念底座，Matt 的代理 workflow 把它落实为任务选择、边界设计、接口约束、测试反馈和评审回收。真正有效的委派由人类保留战局判断，并把战局拆成可执行、可验证、可改进的战术任务。下一步，后续章节会说明这些战略判断如何被写进 skill、状态记录、队列和评审系统，最终回到第一章的乘数公式。

3 teach skill：把学习变成有状态的工作流

3.1 问题：学习代理很容易退化成资料搬运工

开发者向 AI 学习时，最常见的失败模式是从一个宽泛主题开始，例如“教我系统设计”或“教我成为高级工程师”。这种请求看似合理，实际会把学习代理推向资料汇总：列概念、列链接、列路线图，然后让学习者自己决定下一步。Matt Pocock 展示 **teach skill** 的关键价值在于，它把学习重新定义为一个有目标、有记录、有反馈的工作流。

这里需要先解释 **zone of proximal development**，下文简称 ZPD。它指学习者独立完成不了、在合适帮助下可以完成的区域。用工程化记号表达，可以把学习任务放在一个区间里：

$$ZPD = \{t \mid C_{\text{独立}}(t) = 0, C_{\text{辅导}}(t) = 1\}$$

其中：

- t 表示一个具体学习任务，例如“用 Git 保存一次可回退的快照”；
- $C_{\text{独立}}(t)$ 表示学习者独立完成任务的能力；
- $C_{\text{辅导}}(t)$ 表示学习者在教师、材料、练习和反馈帮助下完成任务的能力。

关键结论

核心定义：**teach skill** 的目标 **teach skill** 的目标是把“我要学一个主题”转成“我要完成一个真实使命”。它先定位学习者的任务、背景和成功标准，再围绕这个使命生成资源、课程、练习、测验和 **learning record**。

这就是本章的局部金字塔：

- 问题：学习代理容易只给信息；
- 核心想法：**mission-first learning**；
- 机制：把学习状态写进本地工作区；
- 证据：**mission.md**、资源、HTML lesson、quiz 和 **learning record**；
- 结论：学习型 skill 的价值来自状态，而状态让下一课有依据。

3.2 想法：先问 mission，再决定 lesson

Matt 在演示中没有让 **teach skill** 直接回答“系统设计是什么”。他把自己假扮成一个 **vibe coder**，描述自己只有基础 CLI、能读一点代码、会用终端，并希望补齐知识缺口以便交付更可靠的软件。这个提示的教学价值在于，它告诉代理“学习为了什么”，而非只告诉代理“主题是什么”。

现场对话

原始提示片段 (00:07:45–00:08:35) **Matt**: I am a vibe coder and I want to fill in my knowledge gap so that I can ship better software.

这句话值得保留，因为它把学习目标压缩成一个可操作使命：交付更好的软件。后续 Matt 进一步把使命具体化为“一个声乐老师想做排课、学生笔记、练习辅助的应用”。于是学习路径的起点变成“这个人要把一个真实应用上线，并且不能害怕破坏它”，抽象知识表退到后面。

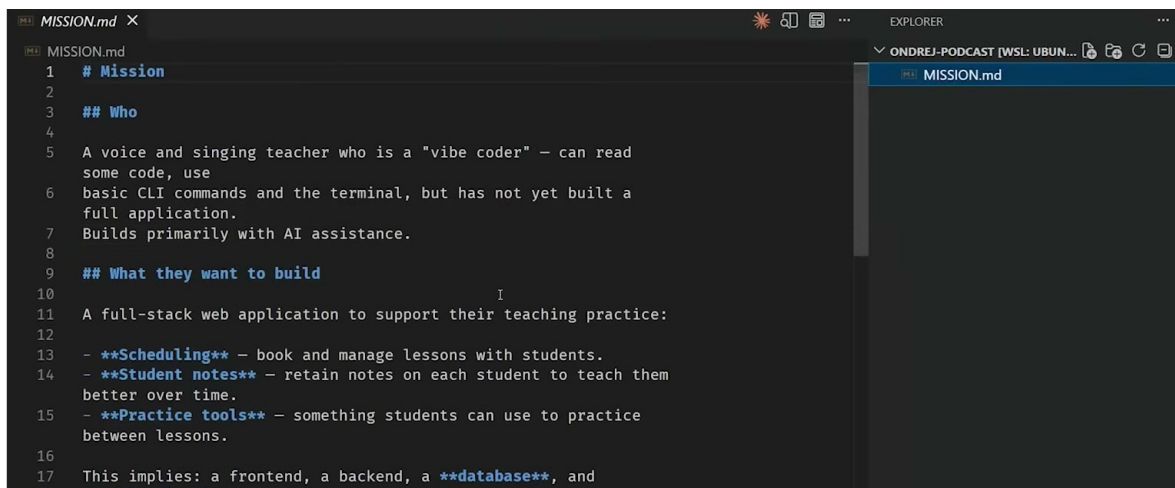
背景知识

背景知识：mission-first learning mission-first learning 可以理解成“先确定读者要穿过哪座桥，再决定讲哪块木板”。对软件学习者而言，使命通常比主题更稳定：主题会从 Git、调试、测试、认证、数据库来回切换，使命则持续约束哪些知识先学、哪些暂缓、哪些练习必须真实上手。

因此，`teach skill` 的第一步是把学习者定位到世界中的一个新位置：他是谁，正在建什么，为什么重要，怎样算成功。这种定位让 agent 像一位记得学生背景的老师，区别于一次性问答工具。

3.3 机制：从 mission.md 到 learning record

`teach skill` 是 `stateful skill`。所谓 `stateful skill`，就是它会在本地工作区保存任务状态，并在后续调用中读取这些状态继续教学。对应的 `stateless skill` 只处理当前输入，不保存学习历史。两者的差异像“临时答疑老师”和“长期带班老师”：前者能回答问题，后者知道学生上节课做错了哪一步。



```
1 # Mission
2
3 ## Who
4
5 A voice and singing teacher who is a "vibe coder" – can read
6 some code, use
7 basic CLI commands and the terminal, but has not yet built a
8 full application.
9 Builds primarily with AI assistance.
10
11 ## What they want to build
12 I
13 A full-stack web application to support their teaching practice:
14 - **Scheduling** – book and manage lessons with students.
15 - **Student notes** – retain notes on each student to teach them
16 better over time.
17 - **Practice tools** – something students can use to practice
18 between lessons.
19
20 This implies: a frontend, a backend, a **database**, and
21 authentication.
```

图 5: `teach skill` 先把学习者身份、目标应用和成功标准写入本地 `MISSION.md`，后续课程围绕这个 mission 展开。¹

在这个案例中，状态可以按职责分成五类，具体文件只是这些职责的载体：

1. 目标状态：MISSION.md 记录学习者身份、要构建的产品、成功标准和当前限制。
2. 材料状态：RESOURCES.md 和 reference 目录保存可信资源、常用命令和关键概念。

¹视频画面时间区间：00:11:14–00:11:30。

3. 教学状态: lessons 目录保存浏览器友好的 HTML 课程, 让学习者沿着一条线性路径前进。
4. 评估状态: quiz 和检查题记录学习者能否主动回忆, 而非只是在阅读时点头。
5. 历史状态: learning-records 记录已经学过什么、测验结果、当前估计水平和下一步方向。

关键结论

机制压缩: 学习记录让下一课有依据 没有 learning record 时, 下一次教学只能重新猜测学习者水平。有了 learning record, 下一课可以沿着“使命、已学内容、错误模式、下一步练习”继续推进。这就是 stateful skill 比一次性提示更适合长期学习的原因。

Open your terminal. This takes 3 minutes and you'll have a real, recoverable project at the end.

1. Make a folder and go into it: `mkdir scheduler && cd scheduler`
2. Start Git: `git init`
3. Create a file: `echo "My app" > notes.txt`
4. Check status: `git status` (see notes.txt in red — untracked)
5. Stage it: `git add .` then `git status` again (now green)
6. Save the snapshot: `git commit -m "First save"`
7. Now **wreck the file**: `echo "ruined" > notes.txt`
8. Undo it: `git restore notes.txt` — open the file. It's back. 🎉

That last step is the whole point. You broke something and got it back in one command. That's the confidence you'll build the app on.

Check yourself

Don't peek at the notes above — pull these from memory. Retrieval is what makes it stick.

图 6: 浏览器里的第一课把 Git 命令拆成可执行步骤, 并通过“Check yourself”引导回忆练习, 体现有状态教学工作流的教学动作。²

这条管线可以概括为:

mission → resources → lesson → quiz → learning record → next lesson

其中 quiz 承担明确教学功能。Matt 明确提到, 测验能增加 storage strength, 也就是记忆被存住的强度。直觉上, 单纯阅读像把文件下载到一个临时目录; 主动回忆像把文件写入带索引的存储系统。学习者回答“哪个命令保存暂存区快照”“git add 做了什么”“破坏文件后如何恢复”时, 知识从被动识别进入可调用状态。

²视频画面时间区间: 00:14:14–00:14:41。

常见误区

常见误区：知识图谱需要线性路径 学习内容可以像知识图谱一样相互连接，学习过程却需要一条线性路径。代理一次性展示整张图，会让新手看到太多入口；`teach skill` 的价值在于按当前 ZPD 切出下一段可走的路。

3.4 例子：Git lesson 如何服务真实应用

演示中的第一课选择从 Git 开始，暂缓宏大的系统设计：创建文件夹、进入项目、初始化、创建文件、查看状态、暂存、更改、提交快照、恢复破坏的文件。这看起来基础，却正好服务“想交付真实应用”的使命。一个缺少版本控制习惯的新手，即便学会数据库和认证，也会在改坏代码时失去安全感。

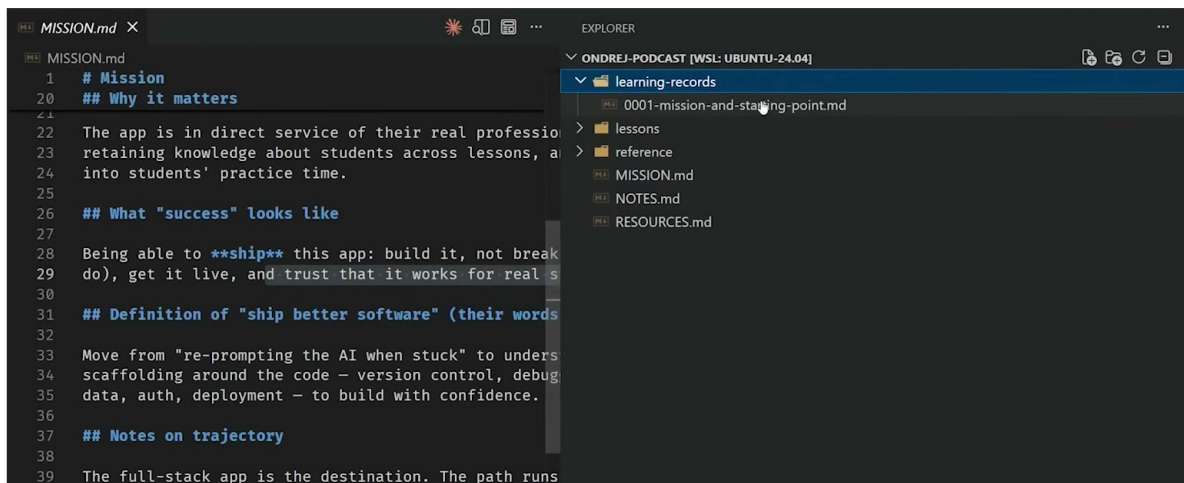


图 7：右侧文件树显示 `learning-records`、`lessons`、`reference`、`MISSION.md`、`NOTES.md` 和 `RESOURCES.md`，说明这个 skill 把学习过程沉淀为本地状态。³

这个例子的证据链很清楚：`mission.md` 把学习者定义为“想做排课应用的声乐老师”；资源和速查表提供参考；HTML lesson 把操作变成可读课程；quiz 用主动回忆巩固命令；`learning record` 记录从 Git 起步的决定、当前估计和后续方向。整个系统把“学习软件工程”压缩成“为了交付应用，今天先获得项目的撤销按钮”。

背景知识

类比：好老师的记忆 一位好老师会记得学生上次卡在换气、节奏还是发声位置；`stateful skill` 也应记得学习者卡在 Git、调试、测试还是部署。状态的用途是让下一次干预更准，保存文本只是手段。

这也解释了为什么 `teach skill` 需要在一个 workspace 中运行。workspace 承担了学习档案袋的角色：课程、记录、资源、练习和下一步计划都在这里积累。

3.5 本章小结

`teach skill` 的核心在于把教学原则工程化：先用 `mission` 定位学习者，再用 ZPD 选择下一步任务；`resources` 提供可信材料，`lesson` 安排线性路径，`quiz` 强化主动回忆，`learning record`

³视频画面时间区间：00:15:49–00:16:10。

保存状态。对 agentic engineering 来说，这个案例说明了一件更大的事：一个好 skill 应当成为可重复运行、能积累状态、能把人类教学判断编码成流程的工作系统。

4 写 skill 的工程判断：procedure skill 优先，谨慎使用 ability skill

4.1 问题：skill 多了以后，真正稀缺的是调用权和上下文预算

Matt 在讨论“好 agent skill 和坏 agent skill 的区别”时，给出的答案绕开模板技巧，落在一个工程判断上：先问这个 skill 应该由谁触发。一个 **procedure skill** 是人类主动调用的流程；一个 **ability skill** 是模型在自己判断需要时调用的能力。二者都能有用，代价结构完全不同。

问题出在 **context window**。每一个允许模型自动发现和调用的 **ability skill**，都需要把描述暴露给模型。描述越多，模型在当前任务里可用的注意力越少。可以把上下文预算写成一个很粗糙的式子：

$$C_{\text{task}} = C_{\text{total}} - C_{\text{history}} - \sum_{i=1}^n C_{\text{ability}_i}$$

其中：

- C_{total} 是模型可用的总 **context window**；
- C_{history} 是当前对话、文件片段、工具结果占用的上下文；
- C_{ability_i} 是第 i 个可自动调用 **ability skill** 的描述成本；
- C_{task} 是真正留给当前问题推理、读代码、写计划和检查结果的上下文。

常见误区

失败模式：100 个 auto-invokable skills 会挤占任务本身 如果团队把所有偏好、规范、流程、检查表都做成自动可调用的 **ability skill**，模型还没开始读任务，就已经被大量描述占走 **context window**。上下文窗口像一张桌面：放满工具说明书之后，真正要处理的零件就没有空间摊开。

4.2 想法：procedure skill 把判断留在人手里

procedure skill 的核心是“可复用的人类判断”。人类决定何时进入某个工作流，例如 **grill me**、PRD drafting、issue splitting、planning、review checklist。模型随后按这个流程追问、组织、拆解或检查。调用权仍然在开发者手里。

关键结论

核心判断：优先把高价值流程写成 **procedure skill** **procedure skill** 适合承载需要人类选择时机的工作：什么时候被追问、什么时候写 PRD、什么时候拆 issue、什么时候进入评审。它把重复出现的专业动作封装成流程，同时保留“是否启动流程”的人类控制权。

Matt 用方向盘类比自己的偏好：开发者应当知道自己拥有哪些技能，知道当前要启动哪个流程，并把模型放在流程内部执行。这个判断很现实。代理擅长填充步骤、生成草稿、检查一致性；任务边界、风险阈值、产品目标和优先级仍然需要人类负责。

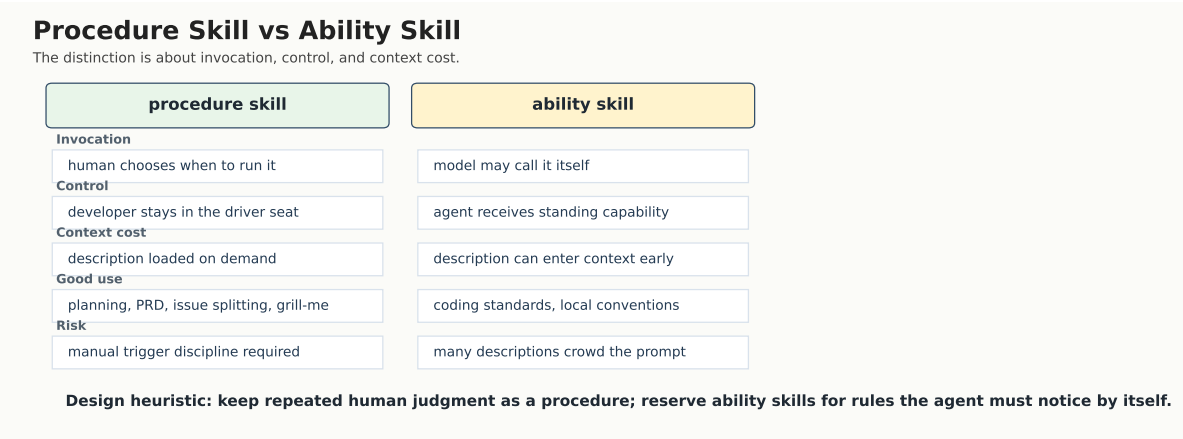


图 8: `procedure skill` 由人选择触发, `ability skill` 由模型自行发现; 核心差异是触发权、控制点和 `context window` 成本。图根据 00:17:21-00:19:13 的 `skill taxonomy` 讨论重绘。

一个实用矩阵如下:

维度	procedure skill	ability skill
触发者	人类显式调用	模型自行判断
控制权	人类选择流程入口	模型选择是否拉取能力
上下文成本	可按需进入, 平时隐藏	描述可能常驻或被提前暴露
适合场景	规划、访谈、PRD、拆 issue、复盘	代码风格、格式规范、小型低争议规则
主要风险	流程写得太空, 无法指导行为	描述膨胀, 模型误触发或注意力被稀释

4.3 机制: `ability skill` 的成本来自自动发现

`ability skill` 的典型例子是 `coding standards`。代理正在写 `React` 代码时, 可能自动拉取团队风格: 避免某些模式、偏好某种状态管理、遵守目录结构。这类能力适合稳定、局部、低歧义的规范。它的问题是, 模型要知道这些能力存在, 就需要看到它们的描述; 描述进入上下文后, 就会和真实任务竞争注意力。

Every auto-invokable ability description competes with task, code, and conversation context.

Every auto-invokable ability description competes with task, code, and conversation context.



图 9: context window 是有限资源, 大量 ability skill 的描述会挤占任务、代码和对话状态。图根据 00:21:13-00:21:51 的“100 descriptions”讨论和 disable-model-invocation 示例重绘。

背景知识

工程直觉：函数抽取与 skill 抽取 Matt 把 skill 类比为代码里的函数抽取：同一套计划方法、访谈方法或评审方法重复出现多次，就可以抽出来做成共享流程。区别在于，函数抽取主要减少代码重复；procedure skill 抽取减少团队工作方式的重复，让更多人按同一套高质量流程推进。

这也是“why procedure skill 优先”的核心答案。流程型技能的描述可以在需要时进入上下文；能力型技能为了自动发现，往往提前占用上下文。流程型技能让人类说“现在进入这个模式”；能力型技能让模型自己判断“我可能需要这个能力”。当工作涉及产品判断、架构取舍、任务拆分和风险控制时，前一种模式更稳。

常见误区

边界提醒: ability skill 应该少而硬 ability skill 适合那些经常用、争议小、判断成本低的规则。比如格式化习惯、命名约定、特定框架的禁止用法。它不适合承载“怎样做产品判断”“怎样决定架构方向”“怎样判断任务是否值得做”这类高语境工作，因为这些工作会消耗大量背景信息，也需要人类承担后果。

4.4 例子：从 grill me 到 PRD 和 issue splitting

Matt 提到的 `grill me` 是一个典型 `procedure skill`。它把模型变成对抗式访谈者，在实现前不断追问：这个计划的边界是什么，哪些决策最有后果，哪些假设还没被检查，哪些风险会在实现时爆炸。这个 `skill` 很短，却能改变模型行为，因为它准确指定了角色、目标和停止条件。

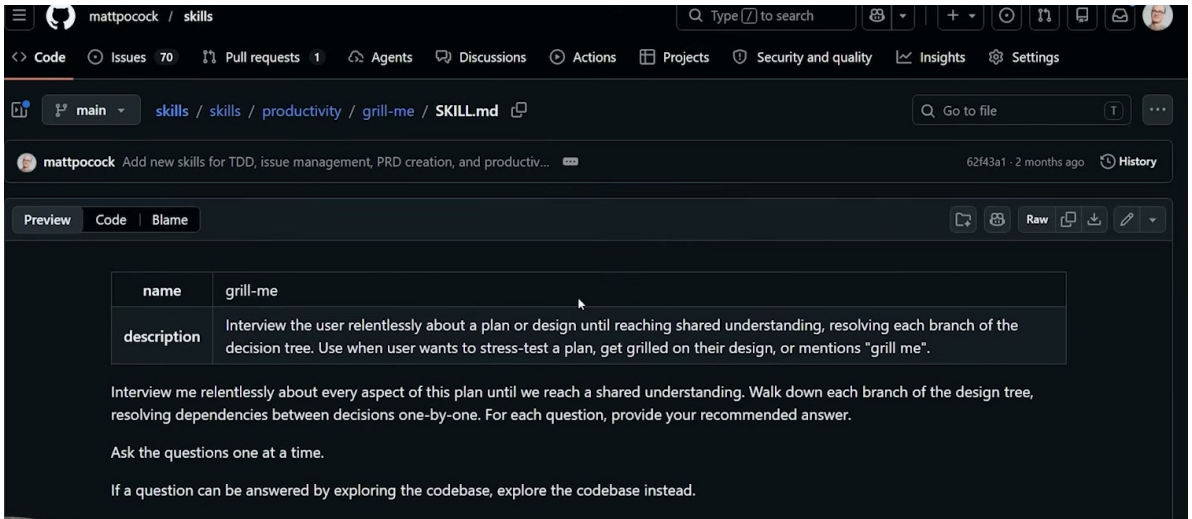


图 10: grill-me 页面展示了一个短小的 `procedure skill`: 用户主动调用它, 让代理围绕计划进行追问和压力测试。⁴

随后, 开发者可以继续调用 PRD drafting, 把共识写成产品需求文档; 再调用 issue splitting, 把 PRD 拆成可执行任务。这个序列体现了 `procedure-first` 的工程品味: 每一步都由人类选择启动, 每一步都生成可检查的中间产物, 每一步都让代理承担具体劳动。

关键结论

控制权原则 当一个 `skill` 会改变任务目标、扩大工作范围、决定优先级或影响评审标准时, 它更应该是 `procedure skill`。当一个 `skill` 只补充稳定规范, 且误触发代价很低时, 它才适合做成 `ability skill`。

这个原则也解释了 Matt 在结尾给出的操作建议: 先回到空白状态, 观察 agent 自己会做什么; 看到基础行为之后, 再逐层加回可理解、可定制、由自己决定启动的 `procedure skill`。重点是恢复观测能力; 工具清单只作为被重新审视的对象。只有先看清基础行为, 开发者才知道哪条流程真正值得封装。

4.5 知识、技能、智慧: `skill` 能封装前两者, 难以替代第三者

Matt 把专业能力拆成 `knowledge/skill/wisdom`。这个三分法很适合解释 `skill` 的上限:

- `knowledge` 是知道什么: 概念、术语、规则、事实、约束。
- `skill` 是做过什么: 反复练习后形成的动作、流程、判断检查表。
- `wisdom` 是何时做、为何做、做到什么程度: 它依赖具体组织、代码库、客户、风险和历史经验。

背景知识

教学价值: `knowledge/skill/wisdom` 的边界 `skill` 最容易封装 `knowledge` 和一部分 `skill`: 例如编码规范、访谈流程、PRD 模板、issue 拆分方法。 `wisdom` 需要在真实语境中反复承受反馈, 例如在某个团队、某个产品、某个代码库里亲自经历成功与失败。

⁴视频画面时间区间: 00:18:18–00:19:13。

这让 `procedure skill` 的位置更清楚：它把已有经验整理成可复用流程，让人类在关键节点继续做判断。好的 `skill` 会提高团队地板线，让较少经验的人也能按成熟流程行动；它不会自动生成资深工程师在真实场景中积累出的全部判断力。

4.6 本章小结

写 `skill` 的第一判断应从触发者开始：该由谁启动这个流程。`procedure skill` 适合封装规划、访谈、PRD、issue splitting 和复盘等高价值流程，因为它保留人类调用权，并把重复的专业动作做成共享 workflow。`ability skill` 适合少量稳定规范，因为它会消耗 `context window`，数量一多就会稀释当前任务的上下文。Matt 的更深层观点是：`knowledge` 和 `skill` 可以被打包，`wisdom` 仍然来自具体处境中的实践与反馈。工程上最稳的路线，是从空白 `agent` 行为开始观察，再逐步加回自己理解、能定制、能负责的 `procedure skill`。

5 AFK workflow: sandbox、queue 与 human-in-the-loop checkpoints

5.1 问题：把 `agent` 放出去干活，首先会遇到边界问题

前几章已经把 `skill` 讲成了可复用的工作方法。本章进入更具体的操作层：当任务已经被拆小，开发者怎样让 `AFK agent` 离开键盘去做事，同时还能保留可控性、可观察性和回收结果的路径。

`AFK agent` 指 away-from-keyboard agent，也就是人类不坐在旁边逐条批准、不实时接管上下文时，被派去完成一个限定任务的代理。它的吸引力很直接：一个人可以同时让多个 `agent` 去探索、修复、评审，再回来看产物。Matt 说自己大量开发已经转向 `AFK` 模式，核心工具是 `Sandcastle`：它把 `agent` 放进 `sandbox`，再让这些 `agent` 在本地或远端并行跑。

这里的第一性问题在于自动化的运行边界。如果 `agent` 可以读写整个用户目录、继承全部环境变量、访问所有网络出口，它就会从工程同事滑向全权限脚本解释器。它可能误改文件，也可能把凭据、API key、环境变量这类 `credentials` 暴露出去。`AFK` 越强，边界越要先设计。

常见误区

sandboxing 与 `credentials` 是 `AFK` 的底线 `sandbox` 的首要身份是权限边界。`AFK agent` 至少应该在受限文件系统、受限网络、最小化凭据、短生命周期 token、可回收工作区里运行。把生产密钥、个人主目录、长期云凭据直接暴露给 `agent`，等于把“试错成本”从代码错误升级成资产泄露。

5.2 例子零：Fable/Cursor 的安全教训要写回 harness

00:35:00–00:42:09 这段是本章和下一章之间的关键桥梁。主持人讲了一个 `Fable/Cursor` 例子：他在设置 `Twitter` 相关 `agent` 时，`Twitter API` 的开发者控制台按钮加载异常；手动换浏览器、关扩展仍未解决后，他把任务交给 `Cursor` 中由 `Fable` 驱动的 `agent`。`agent` 使用 `Cursor` 内置浏览器，登录控制台后开始点击，创建并复制 `API keys`。主持人随即强调，这种做法不适合生产应用。

常见误区

`API key` 示例的真正风险 这个例子展示了 `computer-use agent` 的威力，也暴露了生产边界。能打开浏览器、登录控制台、点击按钮、创建和复制密钥的 `agent`，已经接近真实操作员权限。生产系统里，这类任务至少需要隔离账号、最小权限、短期密钥、审计日志、人工确认和事后轮换策略。把“它能做”直接翻译成“可以让它在生产环境做”，会把 `agentic workflow`

变成凭据事故生成器。

Matt 后面的回应把这个故事从模型炫技拉回工程系统。人仍然要判断任务是否值得做，并在结束时做安全测试。更重要的是，如果一个模型发现了深层安全问题，团队至少学到两件事：模型能力确实有价值；代码库里也存在这类安全缺口。合理反应应当是写入一个可重复机制：

SecuritySignal → TargetedReviewJob → NewCheckOrSkill → Harness_{t+1}

00:39:12–00:39:58 里，Matt 提出可以让一个 cron job 每天检查仓库里的新区域，执行安全 review。这个观点很锋利：深层漏洞未必只能靠最贵、最新的模型发现；如果团队知道该看哪里，并给出正确 prompt 或正确 harness，较便宜的模型也可能找到有价值的问题。换成工程语言，安全审查的质量来自三个因子：

- **Targeting:** 审查指向高风险区域，例如鉴权、权限、密钥、外部 API、数据删除和生产配置。
- **Harness:** 审查 prompt、上下文窗口、代码索引、测试命令和报告格式足够清楚。
- **Feedback:** 发现的问题会变成测试、规则、skill、CI job 或复盘记录。

00:41:38–00:42:05 的自改进系统说法给出了总原则：软件工程一直在做这件事。团队写测试，是为了以后能检查自己的代码；团队做人工 review，是为了确认事情按预期发生；团队重构，是为了以后更容易改变代码。agent 发现安全问题后，也应该触发同样的工程反应：多做一点检查，把偶然发现沉淀为下一轮 harness。

5.3 想法：把 agentic work 优先看成 queue

围绕 agent 自动化，社区常讲 loop：给 agent 一个提示，让它反复运行，直到任务完成。这个模型有用，尤其适合描述一个 runner 的内部循环。不过 Matt 在这里做了一个更贴近软件团队的修正：真正的工作组织更像 queue。

queue 是任务队列：issue、bug report、feature request、review item 都进入队列；开发者、agent、CI job 都是从队列里取任务的节点。这个模型更符合现实开发。团队每天面对的是持续进入的新 issue，以及多个 worker 按优先级、标签和权限取任务的过程。

现场对话

原始概念锚点（00:45:21–00:47:10） **Matt:** “queues, not loops.”

Matt: “we’re really just talking about AFK agents.”

这段短句的价值在于压缩了工作流的核心：loop 说明一个 agent 可以重复执行；queue 说明谁来排队、谁来取活、什么时候回到人类视野、何时从队列里移除。可以把两者的关系想成餐厅厨房：单个厨师切菜、炒菜、装盘时有自己的小循环；整家餐厅靠订单队列运转，订单有优先级、状态、负责人和交付点。

背景知识

loop、queue 与 workflow automation 的关系 loop 是执行模式，描述“一个 worker 怎样反复行动”。queue 是组织模型，描述“多个 worker 怎样从同一批待办事项里取活”。workflow automation 则是连接器：标签、CI、webhook、issue 状态、PR 状态把队列里的任务送到合适的 worker 手里。

5.4 机制：从 issue label 到 explore、implement、review

Matt 展示的机制可以拆成一条从左到右的流水线。先有一个 issue 或 bug report 进入队列；随后通过标签触发 agent 进入某个阶段；agent 产出结构化结果；系统再决定进入下一阶段，或者把任务交还给人。

一个简化的操作流如下：

issue → explore → implement → PR → review → human checkpoint → merge

其中每个阶段的含义都很具体：

- **issue**: 队列里的任务单元，可以来自人工创建、线上 bug report、遥测告警或产品待办。
- **explore**: agent 先调查问题是否真实、是否可修、风险多大、需要哪些文件或测试；这个阶段适合低权限、只读或近似只读的 sandbox。
- **implement**: 当探索结果足够明确，开发者或自动规则给 issue 加上实现标签，例如 `agent implement`；agent 在隔离工作区里提交改动。
- **PR**: 改动回到版本控制系统，以 pull request 的形式接受机器和人的检查。
- **review**: review agent、测试、类型检查、lint、构建和人工评审共同决定产物是否可进入下一步。
- **human checkpoint**: 人类在某个节点检查、批准、要求重跑或补充约束。
- **merge**: 任务从队列里移除，相关经验进入后续改进。

关键结论

queue-based AFK work 是本章的操作核心 成熟 AFK 工作流的关键动作，是把任务拆成可排队、可标记、可回收、可评审的单元。队列负责组织工作，sandbox 负责限制执行边界，review 和 human-in-the-loop checkpoints 负责把结果重新接回工程系统。

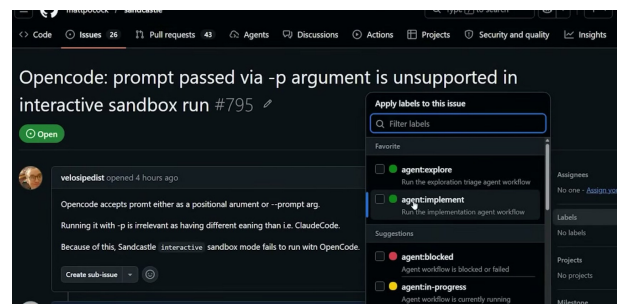
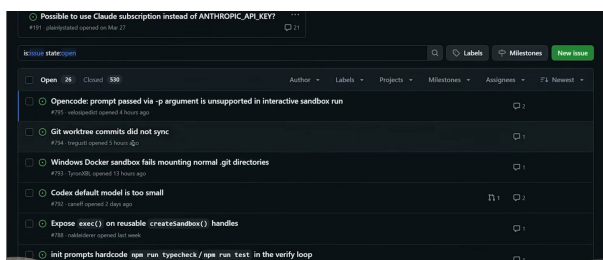


图 11: 左图是 Sandcastle 仓库的 open issue queue; 右图展示 `agent:explore`、`agent:implement`、`agent:blocked` 和 `agent:in-progress` 标签。标签把自然语言任务变成系统可触发的队列信号。⁵

这个模型解释了为什么 issue label 很关键。标签承担的是 workflow 开关功能，分类只是表层效果。比如 `agent explore` 可以触发一次调查；探索报告返回后，开发者再决定是否加 `agent implement`；实现完成后，PR 进入 review 阶段。标签让“任务状态”从聊天里的自然语言变成系统可执行的信号。

⁵视频画面时间区间：00:45:19–00:46:06。

5.5 例子一：Sandcastle 把 AFK agent 放进 sandbox

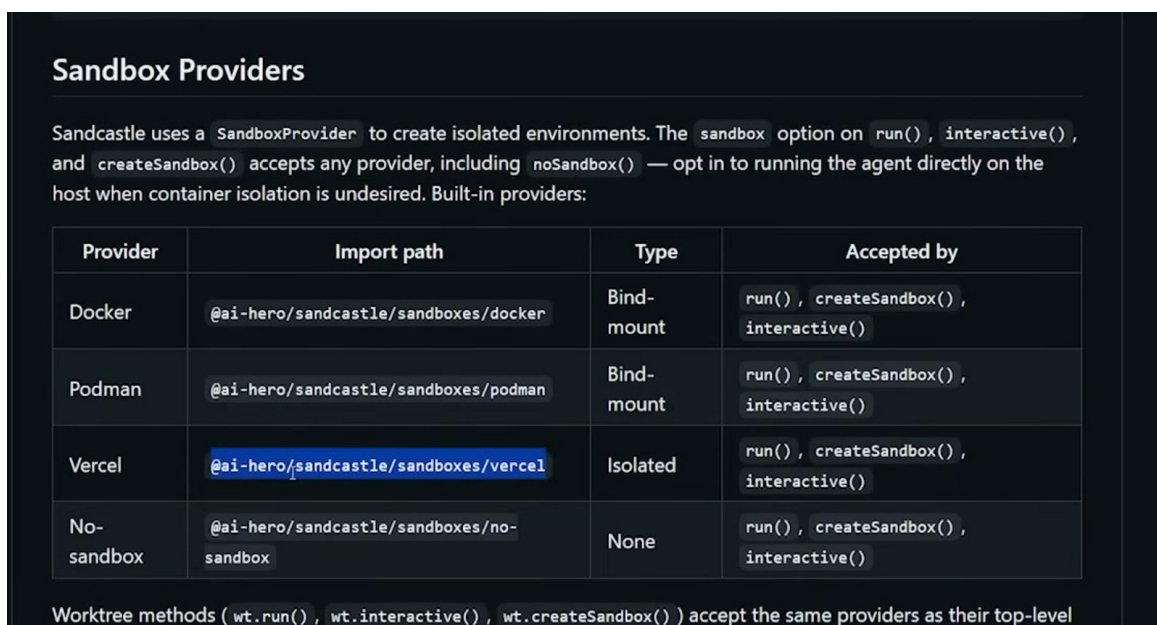
在 00:25 左右, Matt 把 Sandcastle 描述为运行 agent 的 sandbox 工具。它可以接入 Docker、Podman 或远端 sandbox provider, 让 agent 在隔离环境中运行, 然后把提交拉回本地工作区。这个设计有两个实际收益。

第一, 隔离降低失败半径。agent 需要读代码、跑命令、改文件; 这些动作如果发生在主机全权限环境里, 风险会急剧扩大。sandbox 让 agent 更像一个临时外包工位: 能接触当前任务需要的材料, 接触不到无关资产。

第二, 并行提高吞吐。多个 AFK agent 可以同时处理不同任务, 人的工作从“陪每个 agent 打字”变成“准备队列、设定边界、检查结果”。如果把单个开发者的 wall-clock 时间记为 T , 同时运行 k 个独立 sandboxed agents 的理想吞吐可以粗略写成:

$$\text{throughput}_{\text{ideal}} \approx k \cdot \text{throughput}_{\text{single}}$$

这个式子只是直觉模型。真实吞吐还会被任务耦合、review 成本、失败率、CI 资源、冲突合并成本压低。AFK 的价值来自“可并行的限定任务”, 不来自无限制地把所有事情同时扔给 agent。



Provider	Import path	Type	Accepted by
Docker	@ai-hero/sandcastle/sandboxes/docker	Bind-mount	run(), createSandbox(), interactive()
Podman	@ai-hero/sandcastle/sandboxes/podman	Bind-mount	run(), createSandbox(), interactive()
Vercel	@ai-hero/sandcastle/sandboxes/vercel	Isolated	run(), createSandbox(), interactive()
No-sandbox	@ai-hero/sandcastle/sandboxes/no-sandbox	None	run(), createSandbox(), interactive()

Worktree methods (wt.run(), wt.interactive(), wt.createSandbox()) accept the same providers as their top-level

图 12: Sandcastle 的 sandbox provider 表把 AFK agent 的运行边界显式化: Docker、Podman、Vercel 和 no-sandbox 对应不同隔离强度, 也对应不同风险假设。⁶

5.6 例子二：GitHub Actions review agents 把检查接进 PR

Matt 还展示了 GitHub Actions 里的 review agent。这个 agent 在 PR 上运行, checkout 分支, 执行本地 review prompt, 检查类型、测试或其他约束, 然后把结构化反馈返回到 PR。它的角色是把重复检查变成 CI 里的常规步骤, 并把结果交给后续评审系统。

这一步把 AFK 工作从“后台跑完一个补丁”推进到“补丁进入团队已有交付通道”。GitHub Actions 的价值在于稳定触发、可追溯日志和标准化结果: 谁触发了 job, 检查了哪个 commit, 失败在哪一步, 输出了什么结论, 都能被后续 review 看到。

⁶视频画面时间区间: 00:25:24–00:26:18。

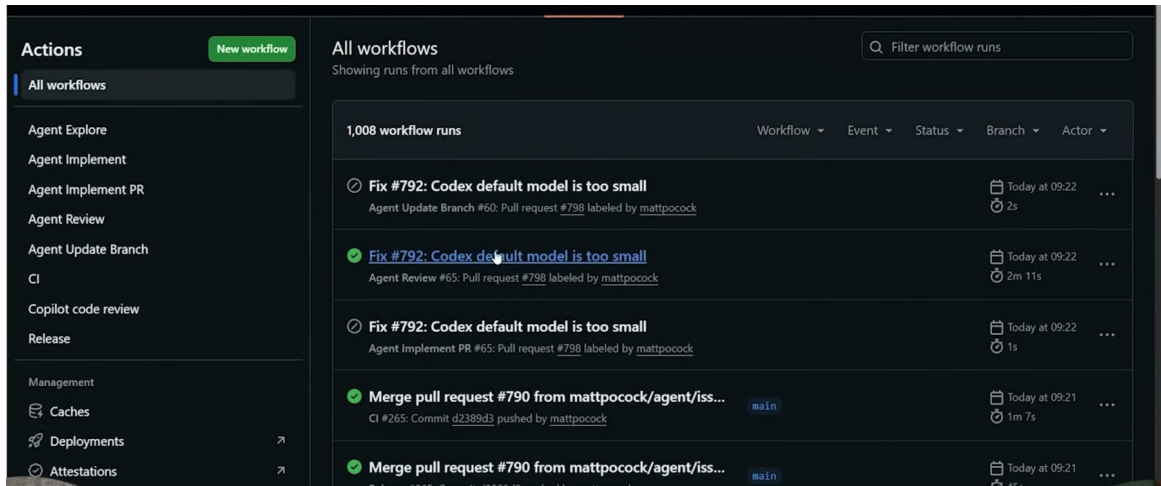


图 13: GitHub Actions 页面显示 Agent Review 已进入标准 workflow run: review agent 被放进 PR 交付通道，保留触发者、分支、状态和耗时等可追溯信息。⁷

把 review agent 放在 PR 上还有一个机械优势: 它天然位于 merge 之前。agent 可以在 sandbox 里大胆尝试, PR 和 CI 则把结果收束到版本控制系统。这样, 人类最终看到的是 diff、测试结果、review agent 结论和必要上下文, 避免面对一段散落在聊天窗口里的过程记录。

5.7 human-in-the-loop checkpoints: 随着信心增长向右移动

human-in-the-loop checkpoints 指人类介入、批准、观察或纠偏的节点。Matt 的关键操作建议是: 这些 checkpoint 应该随着系统成熟逐步向右移动。也就是说, 低信任阶段, 人类可能在 issue 刚出现时就要判断; 更高信任阶段, agent 可以先探索, 带着结构化结果回来; 再成熟一些, agent 可以实现并发起 PR; 对低风险变更, review agent 可能先做机器评审, 再把可点击的决策交给人类。

这里的“向右”是流水线上的位置变化:

bug report → human → explore → implement

可以逐步演进为:

bug report → explore → implement → review → human → merge

这一步减少的是人类在低信息状态下的摩擦, 同时保留人类判断。早期 checkpoint 让人面对原始 bug report; 右移后的 checkpoint 让人面对 bug report、代码库探索、修复方案、测试结果和 review 摘要。Matt 提到的“one button click away”正是这个意思: 人类看到的输入更丰富, 决策更接近最终动作。

⁷ 视频画面时间区间: 00:26:21–00:26:55。

Queue-based AFK workflow

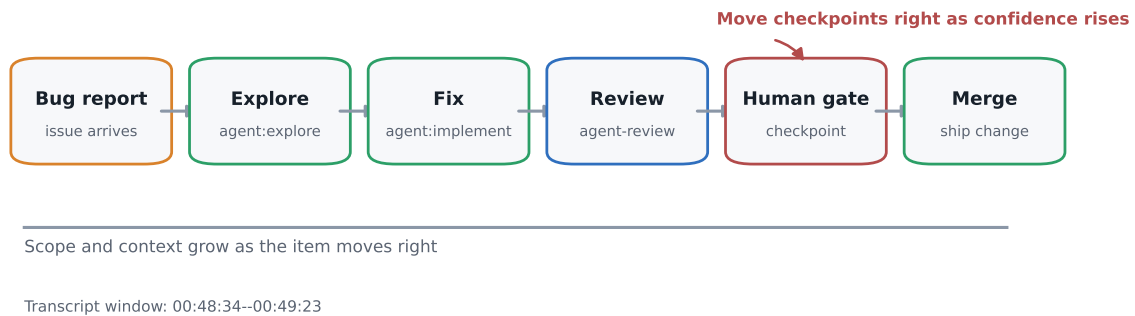


图 14: AFK queue 工作流把 bug report 逐步推进为 explore、fix、review、human gate 和 merge。图根据 00:48:34–00:49:23 的 checkpoint 讨论重绘。

常见误区

checkpoint 右移需要证据，不能靠乐观。只有当任务边界清楚、sandbox 隔离可靠、测试覆盖足够、review agent 输出稳定、失败有回滚路径时，checkpoint 才适合右移。否则右移只是把风险推迟暴露。AFK 工作流的成熟度来自证据：日志、测试、PR diff、review 报告、失败样本和人工抽检。

5.8 操作 takeaway：把 AFK 做成一套可回收的生产线

本章的 workflow mechanics 可以压成五个动作。

1. 先把任务变成队列单元：每个 issue 有清楚范围、输入、预期输出和停止条件。
2. 用标签驱动阶段：explore、implement、review 这类标签负责触发对应 worker。
3. 让 agent 在 sandbox 中运行：文件、网络、凭据、时间和计算资源都要有边界。
4. 让 GitHub Actions 或类似 CI 承接 review agent：机器检查进入 PR，避免留在临时聊天里。
5. 根据证据移动 human-in-the-loop checkpoint：系统越稳定，人类越靠近最终批准点；一旦失败率升高，checkpoint 就向左退回。

这种机制的真正优势是可回收。一次失败会同时暴露代码问题和工作流问题：哪个 label 太粗、哪个 sandbox 权限过大、哪个 review check 缺失、哪个 checkpoint 放得太靠右。队列让任务可见，sandbox 让失败可控，review 让质量可检查，checkpoint 让人类仍然掌握节奏。

5.9 本章小结

AFK agent 的核心价值是并行完成限定任务；queue 是组织这些任务的主模型；loop 只是 worker 内部可能采用的执行模式。Fable/Cursor 的 API key 例子提醒读者，computer-use agent 的能力越强，生产凭据、审计和安全测试越要先设计。Sandcastle 展示了 sandboxed agents 的运行方式，GitHub Actions 展示了 review agents 怎样接入 PR 流程。一个可落地的操作链路是：

issue 进入队列，标签触发 explore，明确后触发 implement，PR 承接 review，human-in-the-loop checkpoint 根据证据逐步向右移动。只要 sandbox、credentials、测试和 review 没有先设计好，AFK 就会把风险并行放大；一旦边界清楚，它会把开发者从实时陪跑转移到队列设计、结果评审和系统改进上。

6 评审、产品与 AX：保留判断力，减少低价值人工成本

6.1 问题：自动化越强，越要保留人的判断位置

第 05 章已经解释了 AFK agent、sandbox、queue 与 human-in-the-loop checkpoints 的运行方式。本章接着处理更棘手的一层：当代理能够探索、修复、提交、甚至建议合并时，人类到底还应该看什么？

Matt 在 00:50:17–00:52:18 的回答把问题压到一个核心点：评审并非只是在 PR 上挑错。评审至少提供两种价值。第一种是闸门价值：拦住危险变更，例如安全事故、源码泄露、行为回归、权限误用。第二种是观测价值：观察这个 agentic system 如何工作，判断 harness 的提示、技能、测试、权限、任务边界是否需要改进。如果团队只看第一种价值，就会很快把“低风险 PR”交给自动合并；如果团队看见第二种价值，就会保留抽样审计和系统反馈。

关键结论

评审对象有两层 评审第一层是 code artifact：这次改动是否正确、可维护、可发布。评审第二层是 producing system：这套 harness、skill、queue、review agent 和自动化策略为什么会产生这样的改动。前者保护生产环境，后者改善未来产出。

这里的未知风险在于“看起来很小”的改动常常依赖隐藏上下文。一个 CSS 对齐问题可能触发可访问性回归；一个内部重构可能改变 bundle 边界；看似普通的文案改动也可能有高风险，例如把退款说明从“可申请退款”改成“保证退款”，会改变用户对服务承诺的理解，并影响客服、合规和争议处理边界。自动化能减少排队、等待和重复检查的成本，治理设计要保留对这些隐藏变量的抽样观察。

6.2 核心判断：把 review 设计成反馈系统，而非人工瓶颈

Matt 接受“human-in-the-loop checkpoints 应该尽量右移”的方向。右移的意思是：人类从一开始查看 bug report，逐渐变成查看 agent 已经完成的探索、修复、验证和风险判定。这里的效率提升来自信息形态升级。人类看到的输入从裸问题升级为经过代理预处理的决策包：问题、原因、diff、测试证据、风险标签、是否可自动合并。

现场对话

评审成本的目标状态（00:49:26–00:49:35）主持人：human 收到的内容可以包含 bug report、codebase exploration、fix 和 review 结果。

主持人：这样就接近“one button click away”，从一整段 debugging session 压缩成一次明确确认。

这句话的概念力量在于，它把 review 从“人重新做一遍调试”改成“人判断一个已整理好的证据包”。证据包越好，人的注意力越能投向 product judgment、risk judgment 和 system judgment。

Review the code and the harness

Human review is also instrumentation for the agent system.

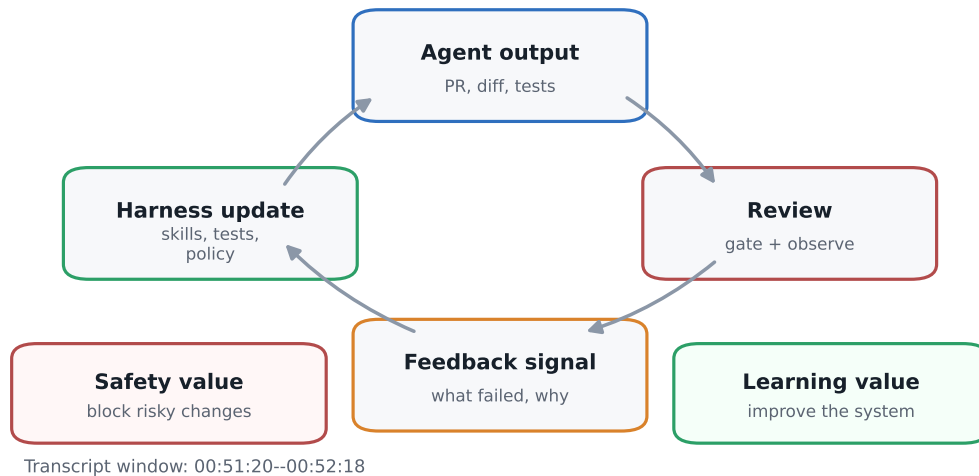


图 15: review 同时检查 code artifact 和 producing system: 一次 review 的输出应当回流到 tests、skills、policy 与 harness。图根据 00:51:20–00:52:18 的反馈系统讨论重绘。

下面这组回流清单是本文作者基于 00:51:20–00:52:18 评讨论做出的工程化综合，用来把“review 同时改善代码和 producing system”的思路转成可检查动作：

- **评审发现：**如果人工或自动评审发现测试缺失、误改边界、说明不清或产品方向偏移，下一轮运行支架要补上对应测试、任务模板、权限限制或评审清单。
- **抽样审计：**如果自动通过或自动合并的样本暴露漏判，下一轮要收紧 gate，增加抽检比例，或把相关任务类型重新拉回人工 checkpoint。
- **回滚数据：**如果发布后出现回滚、热修、告警或用户反馈，下一轮要把失败模式写进测试、监控、回滚脚本和任务边界。
- **产品信号：**如果用户访谈、使用行为、支持工单或销售反馈改变了问题优先级，下一轮要调整 issue 队列、验收口径和代理可执行任务范围。

这组清单的直觉很简单：每次 review 都应该产生两类输出，一类是“这次能不能过”，另一类是“下次怎样让系统少犯同类错”。后者才是 agentic engineering 的复利。

6.3 机制：自动合并需要风险分层、抽样审计与回滚计划

如果团队要减少人工 checkpoint，就需要明确自动化的边界。一个粗糙策略是按任务类型分层：

- **低风险候选：**纯文档修正、无行为变化的格式化、受测试覆盖的内部重命名、小范围 UI 文案调整。
- **中风险候选：**局部重构、依赖升级、影响一条用户路径的 UI 变更、带迁移脚本的数据结构调整。
- **高风险保留人工：**鉴权、支付、隐私、数据删除、权限模型、生产配置、安全边界、跨模块架构改动。

自动合并策略的最低要求是“能解释、能抽样、能回滚”。能解释，表示 review agent 必须给出它为何判定为低风险的依据，例如 diff 范围、测试结果、触达模块、生产影响。能抽样，表示即使系统说“可自动通过”，人类也要按比例检查样本，尤其检查那些被模型判定为 trivial 的变更。能回滚，表示发布策略要有 feature flag、快速 revert、监控告警和责任人。

常见误区

自动合并的真实风险 自动 merge policy 是一套治理协议，远比一个开关更复杂。缺少 audit sampling 时，团队不知道自动 gate 的漏判率；缺少 rollback plan 时，小错误会变成生产事故；缺少反馈记录时，AI 评审器本身无法变得更可靠。

下面的自动化范围模型同样是本文作者的治理综合，用来解释“checkpoint 右移”需要哪些证据支撑：

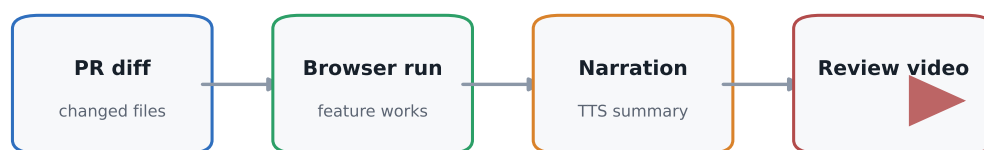
$$\text{AutoMergeScope}_{t+1} = f(\text{EvidenceQuality}_t, \text{FalseNegativeRate}_t, \text{BlastRadius}_t, \text{RollbackSpeed}_t)$$

其中 FalseNegativeRate 是最危险的变量：它代表“系统说没问题，人类抽样或生产事实证明有问题”的比例。只要这个数无法估计，自动合并范围就不能扩张得太快。这里的严厉结论很直接：没有抽样数据的自动合并信心只是感觉。

6.4 例子与证据：更丰富的 review artifacts 会改变人的工作方式

Matt 在 00:53:20–00:54:04 提到一种前端评审形态：AI 对每个前端变更录制 walkthrough video，展示代码和界面变化，再用 text-to-speech 覆盖讲解。这个例子不重要在技术炫技，重要在信息设计。它说明 review artifact 可以从 diff 页面扩展成面向人的说明材料。

AI-generated PR walkthrough



Goal: compress manual review effort without hiding evidence.

The artifact should show the change, the runtime behavior, and a concise explanation.

Transcript window: 00:53:20--00:53:44

图 16: AI-generated PR walkthrough 的价值在于把 diff、浏览器运行结果和讲解压缩成一个面向人类评审者的证据包。图根据 00:53:20–00:53:44 的设想重绘。

一个高质量 review artifact 至少应该回答五个问题：

- **What changed?** 改了哪些文件、路径、组件、接口、状态机或数据表。
- **Why changed?** 对应哪个 issue、用户问题、产品假设或错误报告。

- **How verified?** 跑了哪些测试、覆盖了哪些路径、有哪些截图或视频证据。
- **What can break?** 可能影响哪些边界条件、权限、性能、兼容性和用户路径。
- **What should improve next?** 这次暴露了哪些 harness、test、doc 或 skill 的改进机会。

这类材料的目标是降低人的判断成本，并保留人的最终责任。对工程负责人而言，最宝贵的事情是迅速理解这次变更的风险、产品意义和系统信号，逐行读完所有 diff 只是其中一种手段。

6.5 产品判断：AI 加速实现，产品方向仍来自真实世界

00:54:39–00:56:40 的产品讨论把 agentic engineering 拉回现实。Matt 对“AI 时代 SaaS 会怎样”的宏大预测兴趣很低，他强调的仍是产品基本功：和客户交谈，理解需求，做出能解决真实问题的 prototype，然后持续删除复杂性。

这里要区分三种工作：

- **实现工作：**把已定义的界面、流程、数据结构和测试做出来。AI 在这里能显著提速。
- **产品发现：**知道用户是谁、痛点是什么、为什么现在值得解决、愿意为哪种结果付费。AI 可以辅助整理访谈和生成原型，真实判断仍要来自实际用户。
- **复杂度削减：**发现哪些功能该移除、哪些路径该合并、哪些选择该消失。Matt 的建议很尖锐：可以让 AI 帮忙问“从 app 里移除什么”，因为产品失败常常来自功能膨胀。

背景知识

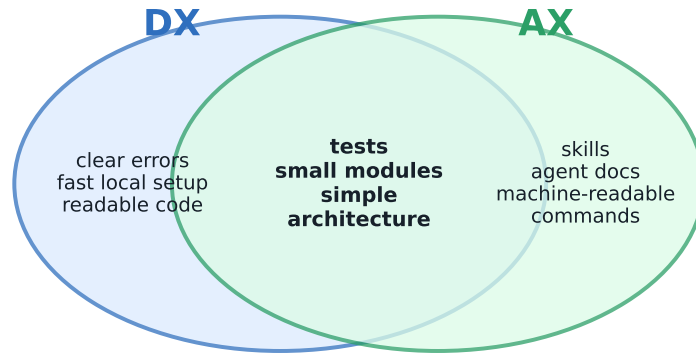
产品判断像导航，代码实现像施工 代理可以让施工队速度翻倍，却不会自动告诉团队城市该往哪里发展。客户访谈、prototype、pricing、retention、support tickets 和真实使用场景提供方向盘；agentic workflow 提供更快的施工机械。

这也解释了为什么“vision”不能被轻易外包。开发者可以不读每个文件，可以不写每行语法，但仍要知道产品为何存在、解决谁的问题、哪些功能该保留、哪些复杂度该移除。

6.6 AX 与 DX：同一个 codebase，两个使用者

Matt 在 00:58:14–00:59:05 引入 DX 和 AX 的区分。DX 是 Developer Experience，即人类开发者在代码库中理解、修改、测试、调试和发布的体验。AX 是 Agent Experience，即代理在代码库中定位任务、理解意图、调用命令、运行测试、解释失败和产出可审查变更的体验。

DX and AX overlap



Developer experience: the codebase is easy for humans to change.

Agent experience: the same codebase exposes safe paths for agents to work.

Transcript window: 00:58:14--00:59:05

图 17: DX 与 AX 在测试、小模块和简单架构上高度重叠; AX 额外强调机器可读命令、agent docs、skills 和稳定验证路径。图根据 00:58:14–00:59:05 的 DX/AX 讨论重绘。

背景知识

把 codebase 想成工作间 好的 DX 像一个工具摆放清楚、标签明确、通道顺畅的工作间; 好的 AX 像这个工作间还配有清晰地图、机器可读指令、稳定命令、可重复测试和权限护栏。人和代理都讨厌杂乱, 只是代理更依赖显式线索。

DX 与 AX 的重叠很大:

- 清晰模块边界帮助人类理解, 也减少代理搜索 token。
- 可靠测试让人类敢改, 也让代理能自证。
- 一致命令让新人少踩坑, 也让 agent 能稳定执行。
- 任务文档和 ADR 帮人类追溯决策, 也给模型提供可引用上下文。
- 小而专注的函数和组件让 review 更快, 也让局部修改更可控。

差异也存在。AX 更需要机器可执行的路径: 脚本名、验证命令、失败恢复步骤、sandbox 约束、fixtures、可解析输出和最小权限。DX 可以依赖团队默契, AX 对默契的容忍度很低。

6.7 团队含义: 资深经验与 AI-native 操作力要合流

00:57:44–01:00:33 的 hiring 讨论容易被误读成 “senior versus junior” 的价值排序。更稳妥的理解是: 资深工程师通常更懂 DX、架构边界、产品取舍和隐性风险; AI-native junior 更可能熟悉最新工具、模型、skills 和代理操作方式。真正的优势来自二者合流: 软件基本功给 AX 提供地基, 实验心态让团队持续发现新的杠杆。

Matt 的强观点是, 纯粹 tactical programming 的价值正在下降。这个判断可以作为警告, 不能扩展成对所有 junior 的否定。junior 可以学战略判断, senior 也必须学习新的 agentic workflow。团队要看的指标应从头衔转向一个人能否持续改善:

- 任务是否更清晰；
- harness 是否更可控；
- review evidence 是否更丰富；
- product decisions 是否更贴近真实用户；
- AX 与 DX 是否共同变好。

6.8 本章小结

第 06 章的核心结论是：agentic engineering 里的 review 是治理系统。它既要防止危险变更进入生产，也要让团队观察代理怎样工作，从而改进 harness。自动合并可以逐步扩大范围，但前提是风险分层、抽样审计、回滚计划和反馈记录同时存在。产品方向仍来自客户、问题、prototype 与复杂度削减；AI 主要提升实现速度和 review artifact 的质量。AX 和 DX 是同一个代码库面对两类工作者的体验，好的工程基本功会同时照亮人和代理的路径。

7 总结与延伸：从空白支架开始，逐步加回 procedure skills

7.1 问题：harness 越厚，越难知道什么真的有效

Matt 的结尾建议听起来激烈：把 skills、plugins、MCP servers、配置文件都清掉，回到 blank slate，再观察 agent 的默认行为。这段建议的重点在于建立可观察的 baseline，危险清理动作不属于这里的操作要求。没有 baseline，开发者很容易把每次成功归因给某条 prompt，把每次失败归因给模型，把真正的问题藏在了一层又一层 instruction、ability skill 和工具配置里。

更稳妥的读法是把 reset 当成受控实验：先隔离一个最小环境，记录当前 skills、plugins、MCP servers 和配置带来的行为，再在空白环境里观察 agent 默认表现。目标是把 baseline、变量和结果分开，避免把 context-window bloat 与模型能力混为一谈。

实验边界也要明确：生产项目保持稳定，实验环境单独承载激进 reset；每次只加回一类 procedure skill 或工具配置，并记录输出质量、可复现性和人工 review 负担的变化。这样既保留 Matt 的 blank slate 建议，也把它落成可观察、可回退、可比较的工程实验。

这条建议背后的问题是 context window bloat。过多自动注入的说明会挤占模型注意力，让 agent 在还没理解任务之前先背了一堆规则。配置越多，因果关系越混；团队越难回答“这次输出变好，究竟来自模型、harness、domain skill，还是 review feedback？”

7.2 核心公式：Agentic Output 来自四个乘数

全篇可以压缩为一个公式：

$$\text{Agentic Output} \approx \text{Model Capability} \times \text{Harness Quality} \times \text{Domain Skill} \times \text{Review Feedback}$$

- **Model Capability:** 模型本身的推理、编码、工具调用和多模态能力。
- **Harness Quality:** 围绕模型的运行支架，包括 prompts、procedure skills、tools、sandbox、tests、docs、CI、queue、review artifacts 和 codebase architecture。
- **Domain Skill:** 人类对业务、工程、产品、教学、设计、安全等领域的知识、技能和智慧。
- **Review Feedback:** 人工与自动检查把错误转化为系统改进的能力。

关键结论

最终行动原则 先建立 blank baseline，观察 agent 自发行行为，再一层一层加回 procedure skills。每加一层都要看它改善了哪一个乘数：model usage、harness quality、domain skill transfer，或 review feedback。

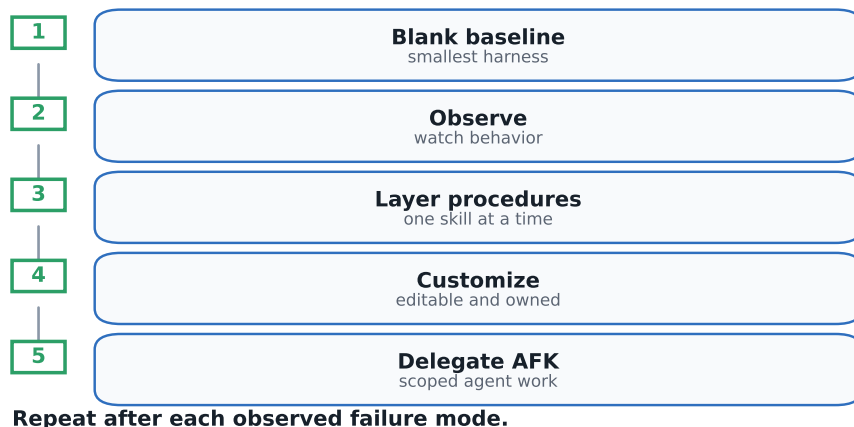
这个公式还有一个残酷含义：任何一个乘数接近零，整体产出都会被拖垮。强模型配上混乱代码库会浪费 token；漂亮 harness 缺少领域判断会造出方向错误的功能；丰富技能缺少 review feedback 会重复犯错；高强度 review 没有可执行改进也会变成慢性疲劳。

7.3 机制：从空白 baseline 到可复用 procedure skills

结尾的操作路线可以拆成五步。

1. **建立 baseline**：在干净 workspace 或隔离分支里运行 agent，只保留必要权限和最小任务说明，观察它如何读项目、改代码、运行测试、报告结果。
2. **记录失败模式**：记录它反复问什么、误解什么、漏跑什么、改坏什么、无法验证什么。这些失败比一次成功更有价值，因为它们指出 harness 缺口。
3. **加回一个 procedure skill**：选择一个人类明确决定触发的 workflow，例如 brainstorming、grill me、PRD、issue splitting、review checklist、release note。一次只加一个，便于观察因果。
4. **定制并度量**：把 skill 改成本项目语言，加入真实命令、路径、测试、风险边界和完成标准。比较加入前后的任务质量、review 耗时、返工率和错误类型。
5. **交给 AFK agent 执行实现**：当任务已经 scoped、验证路径明确、rollback plan 存在时，把实现交给 AFK agent，让人类把注意力放在 vision、product judgment 和 feedback loop 上。

Closing action steps



Transcript window: 01:00:49--01:02:02

图 18：结尾行动步骤可以压缩为五步：blank baseline、observe、layer procedures、customize、delegate AFK。图根据 01:00:49–01:02:02 的收尾建议重绘。

现场对话

结尾行动建议（01:00:59–01:02:02）**Matt**：“go back to a blank slate”，先观察 agent 做什么。

Matt：再把自己决定需要的 procedure skills 一层一层加回去，ability skills 先保持克制。

Matt：把 implementation 尽量委派给 AFK agent；“AFK is just incredible”。

这段话的重点是实验顺序。先观察，再加层；先理解失败，再写规则；先由人决定 workflow，再让 agent 执行实现。这样 procedure skill 承载人类判断，自动注入噪声的风险会下降。

7.4 把个人实践变成团队技能

Matt 在前文区分了 knowledge、skill、wisdom。知识可以写成解释，技能可以写成流程，智慧来自具体情境中的判断。procedure skill 的价值在于把前两者打包，并给第三者留下入口。一个团队反复做 PRD、issue 拆分、架构评审、性能排查、incident 复盘，就可以把这些流程沉淀为可调用技能。

背景知识

从个人习惯到团队 skill 一个资深工程师脑内的“好习惯”像手工师傅的动作：新手看得到结果，却看不清顺序。procedure skill 把顺序写出来：先问哪些问题、检查哪些文件、运行哪些命令、遇到风险怎样升级、完成后怎样复盘。团队成员可以复用它，也可以用 review feedback 改进它。

把实践变成团队 skill 时，要保留三个层次：

- **调用条件**：什么时候用这个 skill，什么时候交给普通 prompt，什么时候需要人类决策。
- **执行协议**：输入、步骤、检查点、命令、输出文件、失败处理、禁止事项。
- **学习记录**：每次使用后出现了什么误判，哪些规则该删减，哪些检查该前移或后移。

这也回应了 ability skill 的风险。让模型自己调用能力很诱人，但每个自动暴露的描述都会占用上下文。procedure skill 让人类保留方向盘：需要它时明确调用，用完后把结果和教训写回 workflow。

7.5 最终综合：战略编程是持续改善乘数

全篇七章其实在讲同一件事：AI 把 tactical programming 变得便宜，战略编程变得更重要。战略编程需要落到一组具体乘数上，停留在抽象口号层面没有用。

- 改善 **Model Capability** 的现实方式，是选择适合任务的模型和工具，避免盲目追逐每个新发布。
- 改善 **Harness Quality** 的方式，是写清任务边界、建立 sandbox、维护 queue、提供 tests、docs、CI 和 review artifacts。
- 改善 **Domain Skill** 的方式，是持续学习业务和工程基本功，把高频流程做成 procedure skills。
- 改善 **Review Feedback** 的方式，是把 review 从单次挑错扩展为系统观测，利用 audit sampling、rollback data 和产品信号更新 harness。

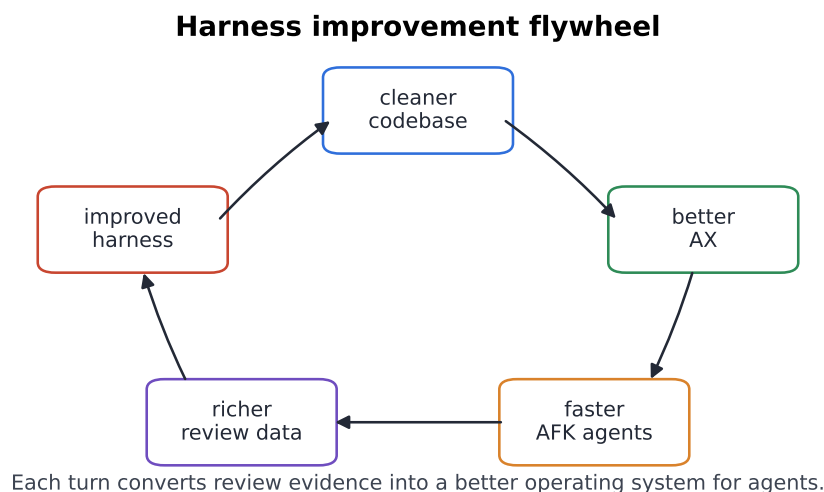


图 19: 最终飞轮: 更清晰的 codebase 改善 AX, 更好的 AX 让 AFK agent 更快, review data 反过来改善 harness。图根据全片综合和 00:58:14–01:02:02 的结尾讨论重绘。

这个飞轮的顺序可以这样理解: 更好的 codebase 带来更好的 DX 和 AX; 更好的 AX 让 AFK agent 更快、更便宜、更少迷路; 更好的 agent output 产生更丰富的 review data; 更好的 review data 反过来改善 harness、tests、docs 和 procedure skills。每转一圈, 团队都在把一次性经验变成可复用结构。

7.6 读者今天可以做的三件事

1. 做一次 **blank baseline** 实验: 在隔离环境里给 agent 一个小任务, 少给额外规则, 观察它的默认行为和失败点。
2. 只加回一个 **procedure skill**: 选择一个最痛的环节, 例如需求澄清、计划审问、issue 拆分、review 汇总。写清触发条件和完成标准。
3. 把实现交给 **scoped AFK agent**: 先把任务切小, 明确文件边界、验证命令和回滚方式, 再让 agent 实现。人类检查证据包, 并把新发现写回 skill 或 harness。

这三件事的价值在于它们都能被观察。观察让团队知道哪一层真的在提高输出, 哪一层只是仪式。agentic engineering 的成熟度, 最终取决于团队能否持续把经验变成 harness, 把 harness 变成更好的 AX, 把 AX 变成更可靠的产出。

7.7 本章小结

Matt 的结尾可以被理解为一反膨胀实验: 先回到 blank slate, 观察 agent, 再按需加回 procedure skills, 并把实现委派给 AFK agent。全篇的主线也在这里闭合: Agentic Output 约等于 Model Capability、Harness Quality、Domain Skill 与 Review Feedback 的乘积。模型会继续进步, 团队真正可控的杠杆仍是运行支架、领域技能、评审反馈和产品判断。